PYTHON DATA VISUALIZATION

What is Data visualization?

Data visualization means showing data using charts, graphs so that people can understand data easily

Ex
- Table - hard to read
- chart - easy to understand

Why Data visualization is important?
- Understand trends
- Compare values
- Find Patterns
- Take better business decisions
- Explain data to managers & clients

Python Libraries for Data visualization

Matplotlib → Static plots → Highly customizable → Steep learning

Seaborn → Statistical visualization

Plotly

Bokeh

Altair

Pygal

Library	Best For	Strengths	Limitations
Matplotlib	Static plots	Highly customizable	Steep learning curve
Seaborn	Statistical visualizations	Easy to use, visually appealing	Limited interactivity
Plotly	Interactive visualization	web integration modern designs	Requires browser rendering
Bokeh	web - based dashboards	Real - time interactivity	more complex setup
Altair	Declarative Statical Plots	concise syntax	Limited customization
Pygal	Scalable SVG charts	High - quality graphics	Less suited for complex datasets

Matplotlib

- what is matplotlib?

matplotlib is a Python library used to draw charts and graphs

Ex

Sales numbers -> Bar chart

Why use matplotlib?

- Easy to learn
- works with Pandas & numpy
- used in real jobs
- Base for seaborn

- Install Matplotlib

```
import matplotlib.pyplot as plt
```

- Import Matplotlib

```
import matplotlib.pyplot as plt
```

`Plt` = shortcut name (used everywhere)

1. Line Chart (Trend)

Used for time-based data

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 20, 25, 40]
plt.plot(x, y)
plt.show()
```

Add labels & title

```
Plt.Plot(x,y)
plt.xlabel("x Axis")
plt.ylabel("y Axis")
plt.title("Line chart Example")
plt.show()
```

2. Bar chart (comparison)
used to compare categories

```
name = ["A", "B", "C", "D"]
values = [20, 35, 30, 10]

plt.bar(name, values)
plt.title("Bar chart Example")
plt.show()
```

3. Pie chart (Percentage)
shows parts & whole

```
labels = ["Food", "Rent", "Travel", "others"]
sizes  = [40, 30, 20, 10]

plt.pie(sizes, labels=labels, autopct="%.1.1f%%")
plt.title("Expense split")
plt.show()
```

4. Histogram (Distribution)
used to see data spread

```
marks = [45, 55, 65, 75, 85, 90, 60, 70, 80]

plt.hist(mark, bins=25)
plt.title("marks Distribution")
plt.show()
```

5. scatter plot (Relationship)
used to see relationship between two numbers.

```
x = [10, 20, 30, 40, 50]
y = [15, 25, 35, 45, 55]

Plt:Scatter (x,y)
plt.title("Scatter plot Example")
plt.show()
```

- Colors, Markers & styles

```
plt.plot(x, y, color="red", marker="o", linestyle="--")
plt.show()
```

Common colors:

red, blue, green, black

Markers:

o, *, x, +

- Multiple Lines in one chart

```
x = [1, 2, 3, 4]
y1 = [10, 20, 30, 40]
y2 = [15, 25, 35, 45]
plt.plot(x, y1, label="Product A")
plt.plot(x, y2, label="Product B")
plt.legend()
plt.show()
```

- Grid (Better Look)

```
plt.plot(x, y)
plt.grid(True)
plt.show()
```

- Figure Size

```
plt.figure(figsize=(8,4))
plt.plot(x, y)
plt.show()
```

save chart as image

```
plt.plot(x,y)
plt.savefig("chart.png")
plt.show()
```

Matplotlib with Pandas

```
import pandas as pd
df = pd.DataFrame({"Year": [2020, 2021, 2022],
                   "Sales": [200, 300, 450]})
plt.plot(df["Year"], df["Sales"])
plt.show()
```

Seaborn

What is seaborn?

Seaborn is a python data visualization library used t
create beautiful charts easily.
It works on top of matplotlib (matplotlib = engine, seaborn

Why use seaborn?

- Less code
- Automatic colors & styles
- works perfectly with Pandas DataFrame
- Best for data analysis & reports

Install & Import

```
pip install seaborn
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

- Load sample dataset (seaborn provides free data)

```
df = sns.load_dataset("tips")
df.head()
```

1 Bar Plot (Average comparison)

used to compare average values.

```
sns.barplot(x="day", y="total_bill", data=df)
plt.show()
```

- use when: average comparison

2. count Plot (Frequency)

Shows count of records

```
sns.countplot(x="day", data=df)
plt.show()
```

- use when: how many records

3. Box Plot (outliers)

used to detect outliers & data spread.

```
sns.boxplot(x="day", y="total_bill", data=df)
plt.show()
```

- use when: outliers

4. Scatter Plot (Relationship)

Show relationship between two numbers

```
sns.scatterplot(x="total_bill", y="tip", data=df)
plt.show()
```

5. Histogram (Distribution)
shows data distribution.

```
sns.histplot(df["total_bill"], bins=10)
plt.show()
```

6. Heatmap (Correlation)
used to show correlation between columns.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True)
plt.show()
```

- use when: correlation analysis

- Hue (category separation)
very important feature in seaborn.

```
sns.barplot(x="day", y="total_bill", hue="sex", data=df)
plt.show()
```

separates data by category

- Style & Theme

```
sns.set_style("whitegrid")
```

Styles:

- darkgrid
- whitegrid
- dark
- white
- ticks

- Pair Plot (Quick Analysis)

Shows relationships of all numeric columns.

```
sns.pairplot(df)
plt.show()
```

- use when : EDA (Exploratory Data Analysis)

- Seaborn vs Matplotlib

Matplotlib	Seaborn
More code	Less code
Manual styling	Auto styling
Basic looks	Beautiful look
Low-level	High-Level

Real-Time use

- Sales analysis
- customer behavior
- outlier detection
- correlation study
- Dashboard visuals

Plotly

what is plotly?

Plotly is a Python library used to create interactive charts.

Interactive means:

- Hover to see values
- Zoom in / zoom out
- Click to hide / show data
- Used in dashboards, reports, web apps

why use plotly?

- Interactive charts
- Beautiful visuals
- Best for dashboards
- works well with Pands
- used in real companies

Install Plotly

ip install plotly

nport Plotly

port plotly. express as px
port Pandas as Pd

tly. express = easy & short code

ine chart (Trend - Interactive)

```
Pd. DataFrame ({ "month": [1, 2, 3, 4, 5],
                 "sales": [200, 300, 250, 400, 450] })
px, line (df, x = "month", y = "sales", title = "Monthly sales Trend")
show()
```

- Hover → see values
- zoom → drag mouse

2. Bar chart (comparison)

```
df = pd.DataFrame({
    "Region": ["North", "South", "East", "West"],
    "Sales": [500, 700, 400, 650]})
fig = px.bar(df, x="Region", y="Sales", title="Sales by Region")
fig.show()
```

- Click legend → hide/show bars

3. Pie chart (percentage)

```
df = pd.DataFrame({
    "payment": ["cash", "card", "UPI"],
    "value": [30, 45, 25]})
fig = px.pie(df, names="payment", values="value", title="Paymen
fig.show()
```

- Hover → % and value

4. Scatter Plot (Relationship)

```
df = pd.DataFrame({
    "Bill": [10, 20, 30, 40, 50],
    "Tip": [2, 4, 6, 8, 10]})
fig = px.scatter(df, x="Bill", y="Tip", title="Bill Vs Tip")
fig.show()
```

- use to see correlation

5 Histogram (Distribution)

```
df = pd.DataFrame({
    "Marks": [45, 55, 65, 75, 85, 90, 60, 70, 80]})
fig = px.histogram(df, x="Marks", title="Marks Distribution")
fig.show()
```

- Shows frequency clearly

Color by category (Very Important)

```
df = px.data.tips()
fig = px.bar(df, x="day", y="total_bill", color="sex",
             title="Day-wise Bill by Gender")
fig.show()
```

color = category separation (like seborn hue)

Plotly with Maps

```
df = px.data.gapminder().query("year == 2007")
fig = px.scatter_geo(df, locations="iso_alpha",
                     size="pop", color="continent",
                     hover_name="country",
                     title="World Population Map")
fig.show()
```

- used in business dashboards
- Save Plotly chart

```
fig.write_html("chart.html")
```